Docker 零基础部署 Web 应用

吴昌庆

2026 年 3 月 21 日

摘要

作为一名开发者，你是否经历过这样的困境：在自己的电脑上写好了一个 Web 应用，测试一切完美。可是当你把代码发给队友，或者部署到云服务器上时，却因为环境不一致而频频报错？本文将带你彻底告别这个痛点。我们将以一个“校园二手交易平台 AI 估价 API”为例，手把手教你如何使用 Docker 实现“所见即所得”的部署能力。

目录

1 引言：为什么我们需要 Docker？ **2**

2 核心概念：Docker 是怎么运转的？ **2**

2.1 镜像 (Image) ——类的定义 2

2.2 容器 (Container) ——类的实例 2

2.3 仓库 (Registry) ——代码托管平台 2

3 前置准备 **3**

4 实战第一步：编写 Dockerfile **3**

5 实战第二步：构建与运行容器 **4**

5.1 构建镜像 (docker build) 4

5.2 运行容器 (docker run) 4

6 实战第三步：如何将容器部署到云平台？ **6**

6.1 推送到仓库 6

6.2 在云服务器拉取并运行 6

7 总结 **6**

1 引言：为什么我们需要 Docker？

在软件开发的世界里，一个应用想要运行，不仅需要**源代码**，还需要庞大的**底层环境支撑**——包括特定版本的操作系统库、编程语言运行时、第三方依赖包等。

在过去，搬迁应用需要在新的服务器上重新装配环境，为了确保环境的正确可能需要不断的调试编译，导致耗费了大量的精力只是做了一件很简单的事情。Docker 的出现，就是为了简化这一点，它允许你把“源代码 + 运行环境 + 配置文件”全部打包进一个标准的盒子里。无论这个盒子被运到 Windows、Mac 还是 Linux 服务器上，内部环境都是恒定不变的。只要服务器装了 Docker，这个集装箱就能无缝运行，也就不用在花费大量的时间在配置环境上了。

2 核心概念：Docker 是怎么运转的？

我们可以用“面向对象编程”的思想来理解 Docker 的三个核心要素：

2.1 镜像 (Image) ——类的定义

镜像是**只读的模板**。可以把它理解为软件的“安装包”或“类 (Class)”。它定义了集装箱里装了什么，但它本身是静态的，不会运行。

2.2 容器 (Container) ——类的实例

当你启动一个镜像时，就创建了一个 **容器**。容器是镜像的运行实例，相当于由“类”`new` 出来的“对象 (Object)”。容器之间互相隔离，互不干扰。这样的好处有很多，比如说在一台机器上要跑很多项目，但是这些项目的很多环境都不相同，就会导致无法在一台机器上同时运行，需要不停的配置环境非常麻烦。但是使用 docker，为每一个项目创建一个容器，每个项目就在自己的容器中跑，因为容器之间的隔离，所以不需要担心容器之间环境的冲突，这样就带来了很大的便利。

2.3 仓库 (Registry) ——代码托管平台

仓库是存放镜像的地方，类似于 GitHub。你可以在本地打包镜像推送到仓库（Push），在服务器上直接拉取（Pull）即可。对于团队协作也是非常有益处的，一个团队只需要在一个仓库里 `Push` 和 `Pull` 即可，极大的降低了团队之间出现环境不匹配的可能性。

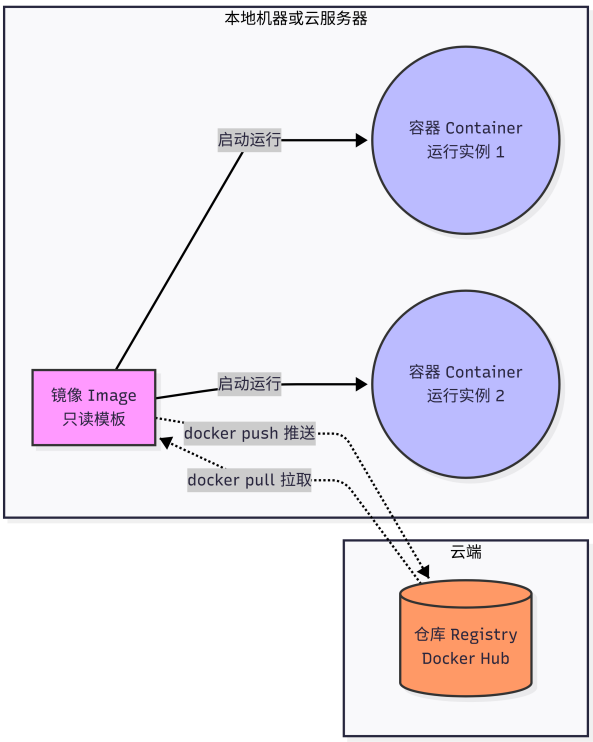

图 1: Docker 镜像、容器与仓库的关系

3 前置准备

在开始敲代码之前，请确保：

1. **安装 Docker Desktop**：前往官网下载并安装。
2. **验证安装**：终端输入 `docker -v`，看到版本号即成功。
3. **准备项目**：假设你有一个基于 Node.js 的项目 `campus-market-ai`（这是我们团队最近在做的项目，比较熟悉拿来举例子）。

4 实战第一步：编写 Dockerfile

`Dockerfile` 是构建镜像的“核心图纸”。在项目根目录下创建一个名为 `Dockerfile` 的文件（无后缀），内容如下：

```
# 1. 指定基础镜像
FROM node:18-alpine

# 2. 设置工作目录
WORKDIR /usr/src/app
```

```

# 3. 复制依赖清单（利用缓存机制加速构建）
COPY package*.json ./

# 4. 安装依赖
RUN npm install

# 5. 复制源代码
COPY . .

# 6. 声明暴露的端口
EXPOSE 3000

# 7. 指定容器启动命令
CMD ["node", "app.js"]
```

Listing 1: Dockerfile 示例

5 实战第二步：构建与运行容器

5.1 构建镜像 (docker build)

在项目根目录执行：

```
docker build -t campus-ai-api:v1 .
```

- `-t`：给镜像起名和打版本号。
- `.`：代表在当前目录寻找 Dockerfile。

5.2 运行容器 (docker run)

```
docker run -d -p 8080:3000 --name my-campus-api campus-ai-api:v1
```

- `-d`：后台运行。
- `-p 8080:3000`：**端口映射**。把电脑的 8080 端口映射到容器内的 3000 端口。现在访问 `http://localhost:8080` 即可看到你的 API。

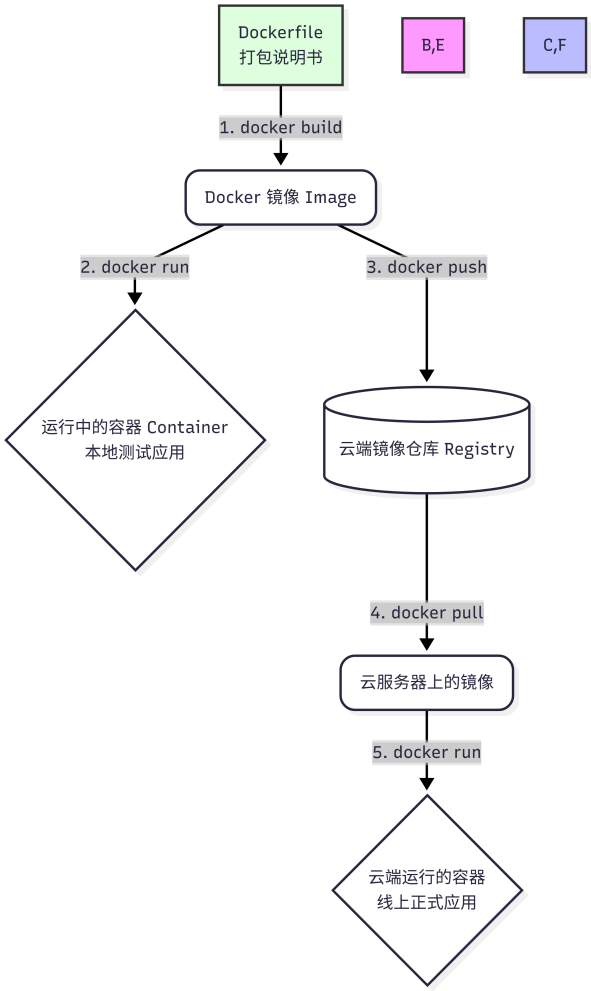

图 2: Docker 部署完整工作流

6 实战第三步：如何将容器部署到云平台？

6.1 推送到仓库

```
docker login
docker tag campus-ai-api:v1 yourname/campus-ai-api:v1
docker push yourname/campus-ai-api:v1
```

6.2 在云服务器拉取并运行

只需在 SSH 连接服务器后执行：

```
docker pull yourname/campus-ai-api:v1
docker run -d -p 80:3000 --name prod-campus-api yourname/campus-ai-api:
    v1
```

通过服务器 IP（默认 80 端口）即可直接访问应用。

7 总结

通过 Docker，我们实现了软件交付的标准化。编写 Dockerfile、熟练掌握 `build` 和 `run`、利用 Registry 分发，是现代 Web 工程师的“部署三板斧”。